GiST, GIN və BRIN

Xüsusi İndeks Növləri - Tam və Ətraflı Bələdçi

Sıfırdan Professional Səviyyəyə - 150+ Səhifə

📌 Bu 3 İndeks Növü Niyə Fərqlidir?

B-Tree

və

Hash

standart məlumat tipləri üçün işləyir: numbers, strings, dates.

Amma bəzi hallarda onlar kifayət etmir:

- 🗺️ Coğrafi məlumatlar: "40.4093°N, 49.8671°E yaxınlığında nə var?"
- 🏷️ Array/JSON: "Bu array-da 'sql' elementi var?"
- 📊 Çox böyük cədvəllər: "1 milyard sətirdə index space problemi"

Bu xüsusi hallarda GiST, GIN və BRIN lazımdır!

1. GiST İndeks - Generalized Search Tree

🗺️ GiST (Generalized Search Tree)

Tam Adı: Generalized Search Tree (Ümumi Axtarış Ağacı)

Yaranma İli: 1995 (University of California, Berkeley)

Əsas Məqsəd: Geometric və mürəkkəb data tipləri üçün axtarış

Strukturu: Balanced tree (B-Tree kimi) amma daha flexible

Əsas Fərq: "Overlapping regions" (üst-üstə düşən bölgələr) konsepti

1.1. GiST Nədir - Fundamental Konsept

GiST **framework**dir - yəni universal template. Müxtəlif data tipləri üçün öz axtarış məntiqini qura bilərsən.

Sadə izah: B-Tree yalnız "böyükdür/kiçikdir" bilir. GiST isə "içindədir", "üst-üstə düşür", "kəsişir", "yaxındır" kimi mürəkkəb əlaqələri anlayır.

💡 Real Həyat Metaforası - Şəhər Xəritəsi:

Klassik B-Tree:

Küçə adları əlifba sırasında

```
A küçəsi
B küçəsi
C küçəsi
...
Z küçəsi
```

GiST İndeks:

Rayonlara və koordinatlara görə

```
Bakı şəhəri
├ Nəsimi rayonu (lat: 40.40, lon: 49.87, radius: 3km)
│  ├ 28 May metro (40.3808, 49.8392)
│  └ Koroğlu metro (40.3856, 49.8289)
├ Nərimanov rayonu (lat: 40.40, lon: 49.87, radius: 2km)
│  ├ Nərimanov metro (40.4034, 49.8722)
│  └ Qara Qarayev metro (40.4056, 49.8917)
└ Səbail rayonu (lat: 40.37, lon: 49.84, radius: 2km)
   └ Sahil metro (40.3650, 49.8356)
```

Axtarış:

"40.40°N, 49.87°E koordinatından 2km daxilində hansı metrolar var?"

GiST cavab verir: Nərimanov metro, Nəsimi metro (geometric calculation ilə)

1.2. GiST İş Prinsipi - Dərin Nəzər

GiST-in əsas ideyası **Bounding Box (Hədd Qutusu)**dur. Hər node bir bölgəni təmsil edir.

📐 GiST Bounding Box Konsepti

```
Təsəvvür edin: 100x100 koordinat sistemi

ROOT NODE (Bütün məkan):
┌─────────────────────────────────────────────────┐
│                                                 │
│  Bounding Box: (0,0) to (100,100)               │
│  "Bütün Azərbaycan"                             │
│                                                 │
└─────────────────────────────────────────────────┘
                    /             \
     ┌──────────────────┐    ┌──────────────────┐
     │ Box 1 (Sol-Aşağı)│    │ Box 2 (Sağ-Yuxarı)│
     │ (0,0)-(50,50)    │    │ (50,50)-(100,100) │
     │ "Bakı və ətraf"  │    │ "Quba-Şəki ətraf" │
     └──────────────────┘    └──────────────────┘
          /        \              /          \
    [Point]    [Point]      [Point]      [Point]
   (25,25)    (30,30)      (75,75)      (80,80)
   "Nizami"  "28May"     "Şəki"       "Quba"

AXTARIŞ PROSESI:
Axtarırıq: (25, 25) koordinatı

Addım 1: ROOT-da yoxla
  - (25,25) Box 1-ə daxildir? ✓ BƏLİ
  - (25,25) Box 2-yə daxildir? ✗ XEYR

Addım 2: Box 1-ə get (Box 2-ni ignore et)

Addım 3: Box 1 içindəki point-lara bax
  - (25,25) === (25,25) ✓ TAPILDI!

Nəticə: "Nizami" stansiyası
Disk I/O: 3 əməliyyat (root + box1 + point)
```

1.3. GiST Daxili Struktur

GiST node-larında saxlanılan məlumat:

```
-- GiST Node Strukturu (Simplified)
struct GiSTNode {
    // Metadata
    bool is_leaf;              // Leaf node yoxsa internal?
    int num_entries;           // Bu node-da neçə entry var

    // Hər entry üçün
    struct Entry {
        BoundingBox bbox;       // Hədd qutusu: (min_x, min_y, max_x, max_y)
        Pointer child;          // Child node və ya actual data
        Predicate pred;         // Axtarış predicate (contains, overlaps və s.)
    }[] entries;
}

-- Məsələn, geometric point üçün:
Entry {
    bbox: (40.40, 49.86, 40.41, 49.88),  // Bölgə
    child: pointer_to_child_node,
    pred: ST_Contains                     // "İçindədir" funksiyası
}
```

1.4. GiST Praktiki Nümunələr

Nümunə 1: Coğrafi Koordinatlar - Metro Stansiyaları

```
-- PostgreSQL-də PostGIS extension lazımdır
CREATE EXTENSION postgis;

-- Cədvəl yaratmaq
CREATE TABLE metro_stations (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    line VARCHAR(50),
    location GEOMETRY(Point, 4326)  -- WGS84 koordinat sistemi
);

-- GiST index yaratmaq
CREATE INDEX idx_metro_location
ON metro_stations USING gist(location);

-- Məlumat daxil etmək (real Bakı metro koordinatları)
INSERT INTO metro_stations (name, line, location) VALUES
('İçərişəhər', 'Qırmızı', ST_SetSRID(ST_MakePoint(49.8356, 40.3650), 4326)),
('Sahil', 'Qırmızı', ST_SetSRID(ST_MakePoint(49.8392, 40.3683), 4326)),
('28 May', 'Qırmızı', ST_SetSRID(ST_MakePoint(49.8392, 40.3808), 4326)),
('Gənclik', 'Qırmızı', ST_SetSRID(ST_MakePoint(49.8467, 40.3783), 4326)),
('Nəriman Nərimanov', 'Qırmızı', ST_SetSRID(ST_MakePoint(49.8722, 40.4034), 4326)),
('Bakmil', 'Yaşıl', ST_SetSRID(ST_MakePoint(49.8264, 40.4147), 4326)),
('Nəsimi', 'Yaşıl', ST_SetSRID(ST_MakePoint(49.8456, 40.4090), 4326)),
('Azadlıq prospekti', 'Yaşıl', ST_SetSRID(ST_MakePoint(49.8554, 40.4028), 4326));

-- SORĞU 1: Konkret nöqtəyə ən yaxın 3 metro stansiyası
SELECT
    name,
    line,
    ST_Distance(
        location,
        ST_SetSRID(ST_MakePoint(49.8500, 40.3900), 4326)  -- Fountains Square
    ) * 111 AS distance_km  -- Dərəcədən km-ə çevirmə
FROM metro_stations
ORDER BY location <-> ST_SetSRID(ST_MakePoint(49.8500, 40.3900), 4326)
LIMIT 3;

-- EXPLAIN ilə yoxlayaq
EXPLAIN ANALYZE
SELECT name FROM metro_stations
WHERE ST_DWithin(
    location,
    ST_SetSRID(ST_MakePoint(49.8500, 40.3900), 4326),
    0.01  -- ~1km radius
);

-- Nəticə:
-- Index Scan using idx_metro_location on metro_stations
--   (cost=0.14..8.16 rows=1 width=32) (actual time=0.045..0.047 rows=2)
```

```
-- Planning Time: 0.112 ms
-- Execution Time: 0.068 ms

-- GiST index istifadə olunur! ✓

-- SORĞU 2: Müəyyən radiusda olan stansiyalar
SELECT
    name,
    line,
    ST_AsText(location) AS coordinates
FROM metro_stations
WHERE ST_DWithin(
    location,
    ST_SetSRID(ST_MakePoint(49.8500, 40.3900), 4326),
    0.015  -- ~1.5km
);

-- SORĞU 3: Polygon daxilində olan stansiyalar
WITH search_area AS (
    SELECT ST_MakePolygon(ST_GeomFromText(
        'LINESTRING(49.83 40.36, 49.86 40.36, 49.86 40.39, 49.83 40.39, 49.83 40.36)',
        4326
    )) AS polygon
)
SELECT
    m.name,
    m.line
FROM metro_stations m, search_area sa
WHERE ST_Contains(sa.polygon, m.location);

-- Nəticə: İçərişəhər, Sahil, 28 May stansiyaları
```

Nümunə 2: IP Address Ranges

```
-- IP aralıqları üçün GiST
CREATE TABLE ip_blocks (
    id SERIAL PRIMARY KEY,
    country VARCHAR(2),
    ip_range inet,
    description TEXT
);

-- GiST index (inet_ops operator class)
CREATE INDEX idx_ip_blocks
ON ip_blocks USING gist(ip_range inet_ops);

-- Azərbaycan IP aralıqları (real data)
INSERT INTO ip_blocks (country, ip_range, description) VALUES
('AZ', '5.197.0.0/16', 'Delta Telecom'),
('AZ', '31.170.0.0/15', 'AzEuroTel'),
('AZ', '37.252.0.0/14', 'Bakcell'),
('AZ', '46.227.192.0/18', 'AzTelecom'),
('AZ', '79.141.160.0/19', 'Azercell'),
('AZ', '81.17.32.0/20', 'Azeronline'),
('TR', '31.145.0.0/16', 'Turkcell'),
('RU', '5.3.0.0/16', 'Rostelecom');

-- Konkret IP hansı ölkəyə məxsusdur?
SELECT
    country,
    description,
    ip_range
FROM ip_blocks
WHERE ip_range >> inet '5.197.100.50';

-- Nəticə: AZ | Delta Telecom | 5.197.0.0/16

-- Subnet-də neçə IP var?
SELECT
    country,
    ip_range,
    masklen(ip_range) AS subnet_mask,
    power(2, 32 - masklen(ip_range))::bigint AS total_ips
FROM ip_blocks
WHERE country = 'AZ';

-- EXPLAIN ilə performans
EXPLAIN ANALYZE
```